

# Daily Paper Review: EvoArena — Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments

Internbot — Automated Research Summary

June 13, 2026

## 1 Paper Summary

### 1.1 Problem and Motivation

Nearly all LLM agent benchmarks (SWE-bench, WebArena, GAIA, AgentBench) assume *static environments*: interfaces, rules, and success criteria are fixed at construction time. In real deployment, however, APIs, workflows, codebases, and user preferences *continually evolve*. Xu et al. [1] identify three critical shortcomings: (1) static snapshots overestimate agent reliability; (2) memory systems that maintain a single latest state suffer *state collapse*—overwriting earlier rules that may still apply to different versions, organizations, or future rollbacks; (3) no existing benchmark simultaneously evaluates *persistent environment evolution* (PE), *implicit change detection* (IC), and *chain evaluation* (CE) over temporally linked task sequences.

A reliable agent must know *what changed* between versions, *what still holds* from prior versions, act correctly under the *current version*, and avoid reusing behaviors tied to *obsolete versions*.

### 1.2 Method

**EvoArena Benchmark.** Environment changes are modeled as *evolution chains*—ordered sequences of progressively evolving releases that preserve the same underlying goal while changing interfaces, rules, or workflows between versions. Three subsets:

- **Terminal-Bench-Evo** (executable workflow evolution): 89 initial tasks converted into chains of 4–5 versions, yielding 441 instances. Evolution types: I/O or protocol changes (49.1%), workspace/module changes (13.4%), CLI/API changes (10.5%), dependency/toolchain changes (8.0%).
- **SWE-Chain-Evo** (software evolution): 50 chains from 12 real GitHub repositories, 493 chain-step instances, mean chain length 9.86. After each step, the *oracle* (not agent) patch is applied to isolate adaptation capability from compounding errors. Cross-step dependency: 29.8% of milestones modify files touched earlier in the chain.
- **PersonaMem-Evo** (social intelligence/preference evolution): 10 persona-level conversations (median 174.7K tokens), 505 preference-inference questions across four types—single-pattern transfer, multi-pattern synthesis, temporal trajectory prediction, and conflict resolution. Five change families: attitude revision (30%), object replacement (30%), conditional shift (13%), attribute shift (13%), temporal-validity shift (14%).

**Dual-Level Evaluation.** (1) *Step accuracy*: average success rate over individual versioned instances. (2) *Chain accuracy*: success across an entire evolution chain (all versions solved for Terminal-Bench-Evo; consecutive milestones from start for SWE-Chain-Evo; all questions in a

preference chain for PersonaMem-Evo). Chain accuracy is strictly harder and tests sustained reliability.

**EvoMem: Patch-Based Memory Evolution.** A lightweight, agent-agnostic memory paradigm with two components:

*Patch recording:* For each non-additive memory update (revision, overwrite, reinterpretation), record a structured patch  $p_t = (\tau_t, C_t^-, C_t^+, r_t, z_t, e_t)$  containing temporal metadata, before/after content, update rationale, semantic summary, and supporting evidence. Purely additive observations remain in base memory without patches.

*Patch-augmented retrieval:* At inference, retrieve from latest memory  $c_{\text{mem}} = R_{\text{mem}}(q, M_T)$ , retrieve relevant patches  $P_q = R_{\text{patch}}(q, P_{1:T})$  (top- $k$ ), and concatenate:  $c(q) = \text{Concat}(c_{\text{mem}}, P_q)$ .

EvoMem is instantiated across four agents: Terminus2 (distilled task-solving knowledge), OpenHands (code context/constraints), A-Mem (semantic notes + graph), and Memento-Skill (global TIP.md file).

### 1.3 Results

**Agents Struggle Under Persistent Evolution.** Average performance across 8 models: Terminal-Bench-Evo 43.6% step / 21.5% chain; SWE-Chain-Evo 27.9% step / 10.0% chain; PersonaMem-Evo 47.3% step / 40.0% chain. The dramatic step-to-chain degradation (43.6%→21.5% in Terminal, 27.9%→10.0% in SWE) demonstrates that solving isolated steps does not translate to reliability across persistent evolution.

**EvoMem Consistently Improves Robustness.** Average improvements: Terminal +2.4% step / **+6.1% chain**; SWE +0.4% step / +2.1% chain; PersonaMem +1.7% step / +3.2% chain. On standard benchmarks: GAIA +6.5%, LoCoMo +3.3%. The chain-level amplification (2–5× the step-level gain) indicates EvoMem is especially effective when success requires preserving evolution-aware knowledge over multiple related states.

**Highlights.** GPT-5.5 on Terminal-Bench-Evo: 62.8%→65.1% step, 31.8%→**45.5%** chain (+13.7pp). Kimi-K2.6 on PersonaMem-Evo: 51.5%→55.5% step, 40.2%→**50.0%** chain (+9.8pp). On GAIA: Deepseek-V4-Pro 70%→80% (+10pp).

**Mechanistic Analysis.** EvoMem helps specifically when retrieved patches are *operationalized* (incorporated into reasoning): no patch uptake yields +2.6%, but patch uptake  $> 0$  yields **+8.3%**. On SWE-Chain-Evo, EvoMem reduces PASS\_TO\_PASS failure rates from 9.09% to 6.32% (−2.77pp), indicating fewer regressions. On PersonaMem-Evo, row-level evidence capture improves by +2.4%, with largest gains on temporal trajectory (+4.4%) and multi-pattern synthesis (+3.5%) questions.

**Negative Results.** EvoMem is not universally positive: GLM-5.1 on PersonaMem-Evo (−2.9%), Gemini-3.1-Pro on SWE-Chain-Evo (−2.4%), GPT-5.5 on temporal trajectory questions (−4.3%). Patch retrieval can introduce ambiguity when identifying the latest valid state.

### 1.4 Limitations

Only three forms of evolution are evaluated (terminal workflows, software repos, user preferences); domains like robotics, scientific workflows, and multi-agent collaboration remain untested. EvoMem is not universally positive—some models degrade with patches, particularly on temporal trajectory questions where outdated patches introduce ambiguity. The benchmark uses oracle-state progression (reference patches applied between steps), which may not

reflect real deployment where agent errors compound. Privacy and adversarial risks arise from persistent memory storing sensitive information.

## 1.5 Relevance to Our Research

EvoArena directly addresses the *evaluation gap* in continual learning for LLM agents (T3). Our method-focused T3 reviews—Continual Experience Internalization [2], Geometry Conflict [3], Abstraction for Continual Learning [4], and the skill evolution papers (SkillOS [5], SkillsVote [6], SkillOpt [7])—developed training-time solutions for catastrophic forgetting and skill accumulation. EvoArena complements these by formalizing the *inference-time* challenge: even a perfectly trained agent fails if its memory system collapses under environment evolution.

The EvoMem paradigm—treating memory updates as structured patches with before/after content, rationale, and evidence—resonates with Memanto’s [8] typed semantic memory and PersonaVLM’s [9] long-term personalization, but adds the critical *version-awareness* dimension. The state-collapse problem directly parallels catastrophic forgetting: where continual learning loses old *parameters*, state-collapse loses old *context*. The git-like versioning analogy suggests that principles from version control (branching, diffing, merging) could be formalized as memory operations for continual agents.

The chain-level amplification finding ( $2\text{--}5\times$  step-level gains from EvoMem) has implications for our RL work: if agents must maintain consistency across evolving environments, reward signals should evaluate chain-level performance rather than isolated steps, connecting to our cross-domain RL interference review [10] where sequential evaluation revealed failure modes invisible in per-domain metrics.

## 2 Follow-up Research Directions

### 2.1 Direction 1: Continual Learning with Evolution-Aware Memory for Self-Evolving Agents

**Gap.** Our Continual Experience Internalization review [2] identified *progressive capability collapse* as a failure mode during agent self-evolution: agents lose previously acquired capabilities as they internalize new experiences. EvoArena reveals the complementary problem: even without capability collapse, agents fail because their *memory* collapses—overwriting context needed for version-aware behavior. Current self-evolving agents (SkillOS [5], SkillOpt [7]) accumulate skills but do not maintain version histories of when/why skills were modified.

**Approach.** Integrate EvoMem-style patch recording into the skill evolution pipeline: when a skill is updated (due to environment feedback or self-improvement), record a structured patch capturing the before/after behavior, the triggering failure, and conditions under which the old behavior remains valid. During skill retrieval, the patch history informs which skill version to activate based on the current environment state. This creates a *versioned skill library* analogous to git branches—multiple skill versions coexist, activated by environment context rather than simply overwritten. Train the version-selection policy via RL with chain-level rewards from EvoArena-style evaluation.

**Feasibility.** The SkillOS/SkillOpt frameworks already maintain skill libraries with metadata. Adding patch recording requires extending the skill update hook to store before/after diffs (minimal code change). The version-selection policy can be trained with GRPO using EvoArena’s chain accuracy as the reward signal. Validation on Terminal-Bench-Evo (which has the largest chain-level gap: 43.6% step vs. 21.5% chain) would directly measure whether versioned skills close this gap. Estimated timeline: 3–4 weeks on existing agent infrastructure.

## 2.2 Direction 2: Memory Evolution as a Reward Signal for Agent RL Training

**Gap.** Current agent RL methods (RAGEN-2 [11], TRACE [12]) use task completion as the reward signal, which captures whether the agent solved the current task but not whether it *maintained consistency* with prior knowledge. EvoArena’s chain accuracy metric reveals that step-level rewards are insufficient—agents achieving 43.6% step accuracy collapse to 21.5% chain accuracy, indicating they solve individual tasks without maintaining coherent state across versions.

**Approach.** Define a *memory consistency reward* that supplements task completion: at each chain step, the reward includes (a) task success (binary), (b) memory coherence (does the agent’s memory correctly reflect what changed and what persists from prior versions), and (c) regression avoidance (does the agent avoid reverting to obsolete behaviors). Formally:  $R_t = R_{\text{task}} + \lambda_1 R_{\text{coherence}}(M_t, P_{1:t}) + \lambda_2 R_{\text{regression}}(a_t, \{a_{\text{obsolete}}\})$ . Train agents with GRPO on evolution chains, where the reward combines step success with memory-evolution quality. The chain-level amplification from EvoMem (+6.1% chain vs. +2.4% step on Terminal) suggests that explicitly rewarding memory management could yield even larger gains.

**Feasibility.** Memory coherence can be evaluated automatically by comparing the agent’s stored memory state against oracle annotations of what should change/persist at each step (available in EvoArena’s construction pipeline). Regression avoidance can be measured by checking whether the agent attempts commands/approaches that were explicitly superseded in prior patches. Integration with existing RL frameworks (OpenRLHF) requires defining the reward function over (trajectory, memory\_state) pairs rather than just trajectory outcomes. The evolution chains provide natural multi-step RL episodes. Estimated compute: training on Terminal-Bench-Evo’s 89 chains with GRPO, feasible on 4 H100 GPUs in 1 week.

## 2.3 Direction 3: Patch-Based Memory for Diffusion LLM Inference Under Prompt Evolution

**Gap.** Diffusion LLMs (LLaDA [13], OPDLM [14]) generate text through iterative denoising, where earlier denoising steps make coarse decisions that later steps refine. When used in agentic settings with evolving contexts (e.g., multi-turn conversations where user requirements change), the model must effectively “track” which prior context is still valid and which has been superseded. Current diffusion LLMs treat each generation independently, with no mechanism for version-aware conditioning. This is analogous to the state-collapse problem EvoArena identifies: the model conditions on the latest prompt without awareness of what changed.

**Approach.** Augment diffusion LLM inference with a patch-conditioned context window: when the prompt/context changes between turns, compute a structured diff (patch) between old and new context, and prepend the patch summary to the denoising input. This provides the model with explicit version-awareness—it knows what changed and can adjust its denoising trajectory accordingly. For block diffusion architectures (like OPDLM’s BD3LM), patches can be injected as a conditioning prefix that attends to all blocks, providing global version context. The approach connects to OPDLM’s rollout-length curriculum [14]: just as rollout length is gradually increased during training, patch complexity could be graduated from simple substitutions to complex conditional changes.

**Feasibility.** Patch computation between prompts is lightweight (text diffing). The conditioning mechanism uses the existing cross-attention or prefix-attention infrastructure in block diffusion models. Training data can be generated by taking multi-turn conversation datasets

and annotating inter-turn changes as patches. Validation: measure generation quality (perplexity, task accuracy) on multi-turn benchmarks where context evolves between turns, comparing patch-conditioned vs. standard conditioning. Implementation requires modifying the OPDLM inference loop to accept patch prefixes—estimated 1–2 weeks of engineering on existing codebase.

### 3 Conclusion

EvoArena establishes the first comprehensive benchmark for evaluating LLM agents under persistent environment evolution, revealing a critical gap between step-level performance (39.6% average) and chain-level reliability (23.8% average) that existing static benchmarks cannot detect. The EvoMem paradigm—treating memory updates as structured, git-like patches rather than state overwrites—provides a principled, agent-agnostic solution that consistently improves chain-level performance with  $2\text{--}5\times$  amplification over step-level gains. The most striking finding is the *operationalization mechanism*: agents benefit from patches only when they actively incorporate retrieved evolution traces into their reasoning (+8.3% with patch uptake vs. +2.6% without). For our research program, EvoArena transforms the continual learning challenge from a purely training-time concern (preventing forgetting) to an inference-time memory management problem (preventing state collapse), and the chain-level evaluation paradigm provides a template for designing RL rewards that capture sustained reliability rather than isolated task success.

### References

- [1] Xu, J., Li, Q., Wu, J., Lan, Y., Li, S.S., Zhou, H., Jiang, B., Wang, L., Wang, J., Luu, A.T., Xiong, C., Park, H.W., Hooi, B., and Hu, Z. “EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments.” *arXiv preprint arXiv:2606.13681*, 2026.
- [2] Liu, J., et al. “Rethinking Continual Experience Internalization for Self-Evolving LLM Agents.” *arXiv preprint arXiv:2606.04703*, 2026.
- [3] Wu, X., et al. “Geometry Conflict: Explaining and Controlling Forgetting in LLM Continual Post-Training.” *arXiv preprint arXiv:2605.09608*, 2026.
- [4] Qian, Y., et al. “Abstraction as a Memory-Efficient Inductive Bias for Continual Learning.” *arXiv preprint arXiv:2603.17198*, 2026.
- [5] Chen, X., et al. “SkillOS: Learning Skill Curation for Self-Evolving Agents.” *arXiv preprint arXiv:2605.06614*, 2026.
- [6] Wang, Z., et al. “SkillsVote: Lifecycle Governance of Agent Skills from Collection, Recommendation to Evolution.” *arXiv preprint arXiv:2605.18401*, 2026.
- [7] Li, Y., et al. “SkillOpt: Executive Strategy for Self-Evolving Agent Skills.” *arXiv preprint arXiv:2605.23904*, 2026.
- [8] Xu, J., et al. “Memanto: Typed Semantic Memory with Information-Theoretic Retrieval for Long-Horizon Agents.” *arXiv preprint arXiv:2604.22085*, 2026.
- [9] Chen, Y., et al. “PersonaVLM: Long-Term Personalized Multimodal LLMs.” *arXiv preprint arXiv:2604.13074*, 2026.
- [10] Liu, J., et al. “A Local Perturbation Theory for Cross-Domain Interference and Recovery in Multi-Domain RL.” *arXiv preprint arXiv:2606.02398*, 2026.

- [11] Xu, H., et al. “RAGEN-2: Reasoning Collapse in Agentic RL.” *arXiv preprint arXiv:2604.06268*, 2026.
- [12] Chen, X., et al. “TRACE: Capability-Targeted Agentic Training.” *arXiv preprint arXiv:2604.05336*, 2026.
- [13] Nie, S., et al. “LLaDA2.0-Uni: Unifying Multimodal Understanding and Generation with Diffusion Large Language Model.” *arXiv preprint arXiv:2604.20796*, 2026.
- [14] Su, Y., et al. “Data-Efficient Autoregressive-to-Diffusion Language Models via On-Policy Distillation.” *arXiv preprint arXiv:2606.06712*, 2026.